Java Course – Class Activity

Exception Handling (3rd August 2026)

Name	Nagireddy Hemanth Reddy
Register No.	192472215

This activity covers four exception types — ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, and NumberFormatException — first with the scenarios given in class (Q1–Q4), and then with four new, self-designed scenarios covering the same four exception types (Q5–Q8).

Q1. ArithmeticException – Student Calculator

Exception Demonstrated: ArithmeticException

Problem Statement

Calculate the average marks of students from total marks and number of subjects. If the number of subjects entered is 0, the program should handle the exception gracefully instead of crashing.

Requirements

- Read total marks and number of subjects.
- Calculate and display the average.
- Handle ArithmeticException using try-catch.
- Display a user-friendly error message when division by zero occurs.

Java Code

```
import java.util.Scanner;

public class StudentCalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter total marks: ");
        int totalMarks = sc.nextInt();
        System.out.print("Enter number of subjects: ");
        int subjects = sc.nextInt();

        try {
            int average = totalMarks / subjects;
            System.out.println("Average marks: " + average);
        } catch (ArithmeticException e) {
            System.out.println("Error: Number of subjects cannot be zero.");
        }
    }
}
```

Sample Input / Output

Sample Input: 450
0

Sample Output: Enter total marks: 450
Enter number of subjects: 0
Error: Number of subjects cannot be zero.

Q2. NullPointerException – Employee Management System

Exception Demonstrated: NullPointerException

Problem Statement

Create an Employee object and display the employee's name. Demonstrate a situation where the employee object is not initialized (null) and handle the resulting exception gracefully.

Requirements

- Create an Employee class with employee ID and name.
- Attempt to access the employee object's data without creating the object.
- Handle NullPointerException using try-catch.
- Display an appropriate message instead of allowing the program to crash.

Java Code

```
public class EmployeeManagementSystem {
    static class Employee {
        int id;
        String name;
        Employee(int id, String name) {
            this.id = id; this.name = name;
        }
    }

    public static void main(String[] args) {
        Employee emp = null;   // employee object not initialized

        try {
            System.out.println("Employee Name: " + emp.name);
        } catch (NullPointerException e) {
            System.out.println("Error: Employee object is not initialized.");
        }
    }
}
```

Sample Input / Output

Sample Input: (no input required — emp is deliberately left null)

Sample Output: Error: Employee object is not initialized.

Q3. ArrayIndexOutOfBoundsException – Student Marks Processing

Exception Demonstrated: ArrayIndexOutOfBoundsException

Problem Statement

Store the marks of five students in an array. Allow the user to enter the index of the student whose mark should be displayed. If the entered index is outside the valid range, handle the exception and display an informative message.

Requirements

- Store marks in an integer array of size 5.
- Accept an index from the user.
- Display the mark corresponding to the entered index.
- Handle ArrayIndexOutOfBoundsException using try-catch.
- Inform the user about the valid index range.

Java Code

```
import java.util.Scanner;

public class StudentMarksProcessing {
    public static void main(String[] args) {
        int[] marks = {78, 85, 90, 65, 72};

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter index (0-4) to view mark: ");
        int index = sc.nextInt();

        try {
            System.out.println("Mark at index " + index + ": " + marks[index]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Valid index range is 0 to 4.");
        }
    }
}
```

Sample Input / Output

Sample Input: 7

Sample Output: Enter index (0-4) to view mark: 7
Error: Valid index range is 0 to 4.

Q4. NumberFormatException – Customer Registration System

Exception Demonstrated: NumberFormatException

Problem Statement

Accept the customer's age as a String and convert it into an integer. If the user enters invalid data (such as alphabets or special characters), the program should handle the exception and display an appropriate error message.

Requirements

- Read age as a String.
- Convert the String into an integer using Integer.parseInt().
- Display the customer's age if the input is valid.
- Handle NumberFormatException using try-catch.
- Display a meaningful message when invalid input is entered.

Java Code

```
import java.util.Scanner;

public class CustomerRegistrationSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter customer age: ");
        String ageInput = sc.nextLine();

        try {
            int age = Integer.parseInt(ageInput);
            System.out.println("Customer age registered: " + age);
        } catch (NumberFormatException e) {
            System.out.println("Error: Please enter a valid numeric age.");
        }
    }
}
```

Sample Input / Output

Sample Input: twenty5

Sample Output: Enter customer age: twenty5
Error: Please enter a valid numeric age.

Q5. ArithmeticException – Bank Loan EMI Calculator

Exception Demonstrated: ArithmeticException

Problem Statement

A bank wants to calculate the monthly EMI (Equated Monthly Installment) for a loan by dividing the loan amount by the number of months chosen for repayment. If the customer enters 0 months, the program should handle the exception gracefully instead of crashing.

Requirements

- Read loan amount and number of months.
- Calculate and display the monthly EMI.
- Handle ArithmeticException using try-catch.
- Display a user-friendly error message when the number of months is zero.

Java Code

```
import java.util.Scanner;

public class LoanEMICalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter loan amount: ");
        int loanAmount = sc.nextInt();
        System.out.print("Enter number of months: ");
        int months = sc.nextInt();

        try {
            int emi = loanAmount / months;
            System.out.println("Monthly EMI: Rs." + emi);
        } catch (ArithmeticException e) {
            System.out.println("Error: Number of months cannot be zero.");
        }
    }
}
```

Sample Input / Output

Sample Input: 120000
0

Sample Output: Enter loan amount: 120000
Enter number of months: 0
Error: Number of months cannot be zero.

Q6. NullPointerException – Library Book Management

Exception Demonstrated: NullPointerException

Problem Statement

A library system needs to display a book's title. Demonstrate a situation where the Book object is not initialized (null) before its title is accessed, and handle the resulting exception gracefully.

Requirements

- Create a Book class with title and author.
- Attempt to access the book object's data without creating the object.
- Handle NullPointerException using try-catch.
- Display an appropriate message instead of allowing the program to crash.

Java Code

```
public class LibraryBookManagement {
    static class Book {
        String title;
        String author;
        Book(String title, String author) {
            this.title = title; this.author = author;
        }
    }

    public static void main(String[] args) {
        Book book = null;   // book object not initialized

        try {
            System.out.println("Book Title: " + book.title);
        } catch (NullPointerException e) {
            System.out.println("Error: Book object is not initialized.");
        }
    }
}
```

Sample Input / Output

Sample Input: (no input required — book is deliberately left null)

Sample Output: Error: Book object is not initialized.

Q7. ArrayIndexOutOfBoundsException – Product Inventory System

Exception Demonstrated: ArrayIndexOutOfBoundsException

Problem Statement

A store keeps the prices of four products in an array. The user enters the index of the product whose price should be displayed. If the entered index is outside the valid range, handle the exception and display an informative message.

Requirements

- Store product prices in a double array of size 4.
- Accept a product index from the user.
- Display the price corresponding to the entered index.
- Handle ArrayIndexOutOfBoundsException using try-catch.
- Inform the user about the valid index range.

Java Code

```
import java.util.Scanner;

public class ProductInventorySystem {
    public static void main(String[] args) {
        double[] prices = {499.0, 999.0, 1499.0, 249.0};

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter product index (0-3) to view price: ");
        int index = sc.nextInt();

        try {
            System.out.println("Price at index " + index + ": Rs." + prices[index]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Valid index range is 0 to 3.");
        }
    }
}
```

Sample Input / Output

Sample Input: 5

Sample Output: Enter product index (0-3) to view price: 5
Error: Valid index range is 0 to 3.

Q8. NumberFormatException – Movie Ticket Booking System

Exception Demonstrated: NumberFormatException

Problem Statement

A movie booking system accepts the number of tickets as a String and converts it into an integer. If the user enters invalid data (such as alphabets or special characters), the program should handle the exception and display an appropriate error message.

Requirements

- Read the number of tickets as a String.
- Convert the String into an integer using Integer.parseInt().
- Display the number of tickets booked if the input is valid.
- Handle NumberFormatException using try-catch.
- Display a meaningful message when invalid input is entered.

Java Code

```
import java.util.Scanner;

public class MovieTicketBooking {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of tickets: ");
        String ticketInput = sc.nextLine();

        try {
            int tickets = Integer.parseInt(ticketInput);
            System.out.println("Tickets booked: " + tickets);
        } catch (NumberFormatException e) {
            System.out.println("Error: Please enter a valid numeric ticket count.");
        }
    }
}
```

Sample Input / Output

Sample Input: three

Sample Output: Enter number of tickets: three
Error: Please enter a valid numeric ticket count.

— End of Submission —